rogii-wellbore-geology-prediction

17th place solution

Rank	17
Team	Falcon
Author	Falcon
Topic ID	733860
Version	Original

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733860
Retrieved with Kaggle CLI on 2026-08-10.

1. Solution overview

The final submission is a weighted average of three models. Model 1 + Model 2 + Model 3 gave **private 6.376 / public 5.853 → 17th place overall**.

```
Model 1  direct prediction     L        GBDT on non-learned estimators, predicting the TVT relative
to the last known TVT
Model 2  integrated slope      Σr     starting from last_known_tvt, integrate the local slope
predicted by a CNN to obtain TVT
Model 3  error correction      δ        predicted systematic error, estimated per 600ft segment

pred = (1−w)·(last_known_tvt + L)  +  w·(last_known_tvt + Σr)  −  λ·δ
       └──── Model 1 ────┘     └── Model 2 ──┘       └ Model 3 ┘

  w = 0.40    weight of the weighted average
  λ(<1200 / 1200–3000 / ≥3000ft) = (1.25, 1.45, 1.60)    correction strength
```

#	Configuration	CV	public LB	private LB
0	GR-likelihood particle filter only	11.08	—	—
1	Model 1	7.09	6.311	7.827
2	Model 1 + 2	6.66	6.130	6.912
3	Model 1 + 2 + 3	**6.32**	**5.853**	6.376

Local validation

Validation was done with a well-level GroupKFold (K=5) on the training data. However, since the difficulty varies a lot from well to well, I did not think RMSE alone was enough to decide whether to adopt a change. I therefore split the wells into four difficulty levels by the RMSE of the GR-likelihood particle filter (the most naive estimator), and evaluated RMSE per level as well.

Level	Definition (per-well RMSE of the PF)	Wells
normal	< 5ft	332
mid	5–10ft	258
high	10–20ft	133
catastrophic	≥ 20ft	50

Why this was needed

RMSE is a quadratic mean, so the catastrophic level — only 6% of the wells — moved the CV score substantially. A change that improved only those wells looked good on CV while improving nothing for the majority of wells, and in practice most changes of the type "fix the hard wells, hurt the normal level" ended up lowering the public LB.

For this reason, the success criterion for an experiment was not only the overall RMSE, but "improvement on every level, and especially a large improvement on the normal level". This turned out to be essential for running a stable improvement cycle when letting an AI agent explore methods autonomously.

Model 1

This is essentially based on the approach published in a public notebook. The target is `TVT − last_known_tvt`, and the model is CatBoost:LGBM:XGB = 2:1:1.

264 features were used, centred on a set of non-learned estimators:

- Estimators that work on a single well (its own GR and type well): GR-likelihood PF, beam search, multi-scale NCC, etc.
- Estimators that borrow labels from neighbouring training wells: PF predictions against a "pseudo type well" built from the labelled TVT-GR of nearby wells, neighbouring structural surfaces, isochores, dip, coverage, etc.

Model 2

Model 1 suffered from a drift that shifts the prediction by a roughly constant amount and depends on the estimators used as features. I therefore built a model that is less exposed to that drift and took a weighted average with Model 1.

- Target: not TVT itself, but the local slope. The predicted slope is integrated again from last_known_tvt to recover the TVT prediction
- Input: one well = one sequence (1ft steps), 74 features. Existing estimators converted into slopes, the velocity and curvature of the trajectory Z, GR and its statistics, disagreement between estimators, etc.

Architecture:

```
input   [74ch × 1024ft]
  ↓  1D Conv ×3 (kernel 32 / 16 / 4, stride 2, 64ch each) → [3ch × 64 × 512]
  ↓  EfficientNet-B0
  ↓  average over the frequency axis to return to a 1D sequence
  ↓  1D Conv head (256 → 128 → 1) → linear interpolation back to the input length
output  [slope over 1024ft]
```

Model 3

Even after the weighted average of Models 1 and 2, an error remained in which the prediction was shifted up or down over a whole segment. So I built a model that predicts that shift per segment by regression and subtracts it from the prediction.

- Target: the shift of the prediction per 600ft segment (the mean residual of the segment)
- Model: LightGBM
- Input: 160 features, mainly the difference between each estimator's prediction and the prediction of Model 1 + 2, plus the spread among the estimators

Because I repeated "analyse the error, then design an improvement" with an AI agent, stacking correction models on top of the residual error became the easiest thing to execute, which is how the solution ended up with this structure.

2. Development with an AI agent

I used a coding agent (Claude Code; Opus 4.8 / Opus 5 / Fable) and ran an exploration of the following scale over roughly two months.

	Count
Experiment scripts	136
Feature-generation scripts	52
Analysis / measurement scripts run by the agent	750+
Agent memories (one finding per file)	232

2.1 Division of work

Based on the split below, my own role was to decide how to explore and to judge whether each method should be adopted. I wrote almost no code myself.

	Owner
Analysing the problem, suggesting ideas	Agent
Formulating the problem, choosing the direction of exploration, proposing hypotheses	Human
Implementation (feature blocks, training scripts, measurement harnesses) and execution	Agent
Interpreting results, adoption decisions, choosing the final submissions	Human
Keeping the experiment ledger and findings, handing them to the next session	Agent

2.2 Defining the direction of exploration

The agent is strong at measurement and implementation, but it tends to close down the search space too early once results come in. After a handful of experiments it concludes "this direction has been fully explored" and tries to move on to the next proposal. Taking its analysis and proposals at face value lets the exploration continue from a wrong conclusion and eventually stall, so it was important that a human understood and judged whether the direction was sound.

For example, right after we found that the main component of the residual was "an error that shifts each well by a constant amount", the agent concluded that "GR, well-level features and the local geometry are all uninformative about this shift", and tried to close the direction down. What had actually been refuted, however, were only two specific estimators — naive GR matching and a regression on aggregated existing features — and not the direction of "predicting the shift" itself. Pushing back and continuing the exploration is exactly what produced Model 3: a direction that had been closed once, and that in the end delivered a real improvement.

Similarly, there was a period where that correction stayed in its most naive form, one constant per well. Pointing out that it had not been explored deeply led to the form that corrects per 600ft segment, which improved the score further. The methods and features that actually worked came from not letting the exploration be closed early and keeping it going.

2.3 What was difficult / did not work well with an AI agent

- Accumulated memory fixes the agent's thinking, and the direction of investigation becomes rigid
- The LLM has a notion of time and cost, and tends to try only cheap methods; towards the end of the competition it starts proposing only simple approaches
- The generated analyses and implementations get too complex to follow, and the human then ends up requesting experiments that are not meaningful

2.4 Summary

Letting the agent handle implementation and measurement raised the number of iterations enormously.

- To increase autonomy, it was important to fix the criterion of what counts as an improvement
- The agent's analyses and proposals should not be taken at face value; the human has to judge the direction and hand it back
- Solving a task as complex as a Kaggle competition with an LLM needed tuning, such as instructing it not to close down investigations too easily
- Taking compute resources into account, it was important to steer the LLM so that the expected value of the exploration is maximised — for instance by telling it not to focus on cheap methods or methods with only marginal gains